1. บทนำ (Introduction)

1.1 ภาพรวมของระบบ (System overview)

Pai Nam Nae เป็นแอปพลิเคชันบนเว็บสำหรับแชร์รถที่เชื่อมต่อผู้ขับขี่และผู้โดยสารที่ต้องการเดินทางไปในเส้นทางเดียวกัน โดยเน้นความปลอดภัยและความสะดวกสบายเป็นหลัก พัฒนาด้วย Nuxt.js เป็น Frontend และ Express.js เป็น Backend โดยใช้ Prisma ORM และฐานข้อมูล PostgreSQL

1.2 วัตถุประสงค์ในการทดสอบ (Purpose of Test)

เพื่อทดสอบการทำงานส่วนของการลบข้อมูล ว่า สามารถดำเนินการลบข้อมูล และตรวจสอบความถูกต้องข้อมูลก่อนที่จะดำเนินการตามขั้นตอนต่อไปได้อย่างถูกต้อง

2. วิธีการทดสอบและกระบวนการทดสอบ (Test methodology and test process)

ผู้ทดสอบทำการทดสอบระดับ User acceptance test (UAT),API test และ Unit test ด้วยวิธีการทดสอบแบบอัตโนมัติ (Automated test) และตรวจสอบการทำงานของส่วนการลบข้อมูล ในกรณีดังต่อไปนี้

(1) ผู้ใช้มี Account ในระบบ
(2) ผู้ใช้ Login ได้
(3) ผู้ใช้สามารถรับ OTP ทาง Email ได้
(4) มี Admin สำหรับลบข้อมูลทดสอบ

3. สภาพแวดล้อมในการทดสอบ (Testing Environment)

3.1 ฮาร์ดแวร์ (Hardware)

CPU: Intel Core i5 12400F
Mainboard: PRO H610M-S
Memory: DDR4 32GB BUS3200
Graphics: NVDIA GeForce RTX 2070 SUPER

3.2 ซอฟต์แวร์ (Software)

- Nodepad++
- Javascript
- Robot-framework

3.3 ผู้ทดสอบ (Human)

นายตะวัน อุตมาน

4. รายละเอียดการทดสอบ (Test scenario and test design)

4.1 คำอธิบาย (Description)

✓ หมายถึง ผ่าน (Pass)

✗ หมายถึง ไม่ผ่าน (Fail)

4.2 ผลการทดสอบ (Test results)

CP353004/SC313004 Software Engineering

ผศ.ดร. ชิตสุธา สุ่มเล็ก **ปีการศึกษา 2/2568**

Test Scenario ID:	UAT016-1	**Project ID:**	PaiNamNae
Test Scenario Name:	UAT - ลบ Account สำเร็จ	**Tested by:**	ตะวัน อุตมาน
UAT Name:	Deletion Story	**Version:**	V1.0
Module:		**Date of Test:**	17/03/2569
Pre-requisite:	- ผู้ใช้มี Account ในระบบ - ผู้ใช้ Login ได้ - ผู้ใช้สามารถรับ OTP ทาง Email ได้ - มี Admin สำหรับลบข้อมูลทดสอบ		
Description:			

No.	Test Case and Steps	Expected Result	Actual Result	Test Result (Pass/Fail)	Remark
1	Delete Account Success: 1. Login เข้าสู่ระบบ 2. ไปที่หน้า Profile 3. กดปุ่ม “ลบบัญชี” 4. กดยืนยันเงื่อนไข 5. กดปุ่มขอ OTP 6. กรอก OTP ที่ถูกต้อง 7. กดยืนยันลบ 8. กด Accept Alert	- ระบบลบบัญชีสำเร็จ - แสดงข้อความยืนยันการลบสำเร็จ - Session ถูกยกเลิก - ไม่สามารถใช้งานบัญชีนี้ได้อีก	- ระบบลบบัญชีสำเร็จ - แสดงข้อความยืนยันการลบสำเร็จ - Session ถูกยกเลิก - ไม่สามารถใช้งานบัญชีนี้ได้อีก	✓	
2	Verify Cannot Login After Deletion: 1. Login เข้าสู่ระบบ 2. ไปที่หน้า Profile 3. กดปุ่ม “ลบบัญชี” 4. กดยืนยันเงื่อนไข 5. กดปุ่มขอ OTP 6. กรอก OTP ที่ถูกต้อง	- ระบบแสดงข้อความ “เข้าสู่ระบบไม่สำเร็จ” - ไม่สามารถเข้าสู่ระบบได้	- ระบบแสดงข้อความ “เข้าสู่ระบบไม่สำเร็จ” - ไม่สามารถเข้าสู่ระบบได้	✓	

	7. กดยืนยันลบ 8. กด Accept Alert 9. ไปที่หน้า Login 10. กรอก Username และ Password เดิม 11. กด Login				

CP353004/SC313004 Software Engineering

ผศ.ดร. ชิตสุธา สุ่มเล็ก **ปีการศึกษา 2/2568**

Test Scenario ID:	UAT016-2	**Project ID:**	PaiNamNae
Test Scenario Name:	UAT - ลบ Account ไม่สำเร็จ	**Tested by:**	ตะวัน อุตมาน
UAT Name:	Deletion Story	**Version:**	V1.0
Module:		**Date of Test:**	17/03/2569
Pre-requisite:	- ผู้ใช้มี Account ในระบบ - ผู้ใช้ Login ได้ - ผู้ใช้สามารถรับ OTP ทาง Email ได้ - มี Admin สำหรับลบข้อมูลทดสอบ		
Description:			

No.	Test Case and Steps	Expected Result	Actual Result	Test Result (Pass/Fail)	Remark
3	Wrong OTP: 1. Login 2. ไปที่หน้า Profile 3. กดปุ่มลบบัญชี 4. กดยืนยันเงื่อนไข 5. กรอก OTP ผิด (เช่น 123456) 6. กดยืนยัน	- ระบบแจ้งเตือน OTP ไม่ถูกต้อง - บัญชียังไม่ถูกลบ - ผู้ใช้ยัง Login ได้	- ระบบแจ้งเตือน OTP ไม่ถูกต้อง - บัญชียังไม่ถูกลบ - ผู้ใช้ยัง Login ได้	✓	
4	Expired OTP: 1. Login 2. ไปที่หน้า Profile 3. กดปุ่มลบบัญชี 4. กดยืนยันเงื่อนไข 5. ขอ OTP 6. รอจน OTP หมดอายุ 7. กรอก OTP เดิม 8. กดยืนยัน	- ระบบแจ้งเตือน OTP ไม่ถูกต้อง - ไม่สามารถดำเนินการลบได้ - บัญชียังไม่ถูกลบ	- ระบบแจ้งเตือน OTP ไม่ถูกต้อง - ไม่สามารถดำเนินการลบได้ - บัญชียังไม่ถูกลบ	✓	

No.	Test Case and Steps	Expected Result	Actual Result	Test Result (Pass/Fail)	Remark
5	Cancel Delete Process: 1. Login 2. ไปที่หน้า Profile 3. กดปุ่มลบบัญชี 4. กดยืนยันเงื่อนไข 5. ขอ OTP 6. กรอก OTP ถูกต้อง 7. กดปุ่ม “ยกเลิก”	- ระบบไม่ลบบัญชี - ไม่มีการเปลี่ยนแปลงข้อมูล - ผู้ใช้ยัง Login ได้ตามปกติ	- ระบบไม่ลบบัญชี - ไม่มีการเปลี่ยนแปลงข้อมูล - ผู้ใช้ยัง Login ได้ตามปกติ	✓	
6	Cannot Delete When Have Route: 1. Login ด้วยบัญชีที่มี route 2. ไปที่หน้า Profile 3. กดปุ่มลบบัญชี	- ระบบแสดงข้อความ “ไม่สามารถลบข้อมูลบัญชีได้เนื่องจาก:” - ไม่สามารถดำเนินการลบต่อได้	- ระบบแสดงข้อความ “ไม่สามารถลบข้อมูลบัญชีได้เนื่องจาก:” - ไม่สามารถดำเนินการลบต่อได้	✓	

Test Scenario ID:	UAT016-3	**Project ID:**	PaiNamNae
Test Scenario Name:	UAT - ทดสอบ checkboxes	**Tested by:**	ตะวัน อุตมาน
UAT Name:	Deletion Story	**Version:**	V1.0
Module:		**Date of Test:**	17/03/2569
Pre-requisite:	- ผู้ใช้มี Account ในระบบ - ผู้ใช้ Login ได้ - ผู้ใช้สามารถรับ OTP ทาง Email ได้ - มี Admin สำหรับลบข้อมูลทดสอบ		
Description:			

No.	Test Case and Steps	Expected Result	Actual Result	Test Result (Pass/Fail)	Remark
7	Verify Data Backup Checkboxes: 1. Login ด้วย User ที่มีข้อมูล 2. ไปที่หน้าลบบัญชี ลองติ๊กเลือก/เอาออก ในช่องข้อมูลต่างๆ (โปรไฟล์, รถยนต์, ฯลฯ) 3. Login ด้วย User ที่ไม่มีข้อมูล 4. ไปที่หน้าลบบัญชี สังเกตช่อง Checkbox	- ช่อง Checkbox ที่มีข้อมูลสามารถติ๊กเลือกและเอาออกได้ปกติ - ช่อง Checkbox ที่ไม่มีข้อมูลระบบจะแสดงข้อความ "(ไม่มีข้อมูลในระบบ)" และอยู่ในสถานะ Disabled กดไม่ได้	- ระบบอนุญาตให้เลือกสำเนาได้เฉพาะข้อมูลที่มีอยู่จริง - ข้อมูลที่ไม่มีจะถูกล็อกไว้ไม่ให้เลือก (Disabled)	✓	
8	Verify Agreement Checkbox: 1. Login เข้าสู่ระบบ 2. ไปที่หน้า Profile 3. กดปุ่ม "ลบบัญชี" 4. สังเกตปุ่ม "ลบ" ด้านล่างสุด (ต้องกดไม่ได้) 5. ติ๊กช่อง "รับทราบและยอมรับเงื่อนไข" 6. สังเกตปุ่ม "ลบ" (ต้องเปลี่ยนเป็นกดได้)	- ปุ่ม "ลบ" จะอยู่ในสถานะ Disabled หากยังไม่ยอมรับเงื่อนไข - ปุ่ม "ลบ" จะเปลี่ยนเป็นสถานะ Enabled ทันทีเมื่อติ๊กยอมรับเงื่อนไข	- ปุ่ม "ลบ" ตอบสนองต่อการติ๊กยอมรับเงื่อนไขได้อย่างถูกต้องตามที่คาดหวัง	✓	

CP353004/SC313004 Software Engineering

ผศ.ดร. ชิตสุธา สุ่มเล็ก **ปีการศึกษา 2/2568**

Test Scenario ID:	API016	**Project ID:**	PaiNamNae
Test Scenario Name:	API - Account Deletion	**Tested by:**	ตะวัน อุตมาน
UAT Name:	Deletion Story	**Version:**	V1.0
Module:		**Date of Test:**	03/03/2569
Pre-requisite:	- ระบบ Backend ทำงานที่ ${URL} - สามารถเรียก API ได้ - มีสิทธิ์ Register ผู้ใช้ - มีสิทธิ์ Login		
Description:			

No.	Test Case and Steps	Expected Result	Actual Result	Test Result (Pass/Fail)	Remark
9	Check Delete Request Infos: 1. เรียก API Register เพื่อสร้างผู้ใช้ใหม่ 2. Login เพื่อรับ ACCESS_TOKEN 3. สร้าง Header (Authorization: Bearer Token) 4. เรียก GET /api/delete-request/check-infos 5. ตรวจสอบ Response	- Status Code = 200 - Response JSON มี key: • user • vehicles • routes • booking	- Status Code = 200 - Response JSON มี key: • user • vehicles • routes • booking	✓	
10	Check Can Delete: 1. Register ผู้ใช้ใหม่ 2. Login เพื่อรับ ACCESS_TOKEN 3. เรียก GET /api/delete-request/check-can-delete 4. ตรวจสอบ Response	- Status Code = 200 - Response JSON มี key canDelete	- Status Code = 200 - Response JSON มี key canDelete	✓	
11	Send OTP: 1. Register ผู้ใช้ใหม่ 2. Login เพื่อรับ ACCESS_TOKEN 3. เรียก POST /api/otp/send 4. ตรวจสอบ Response	- Status Code = 200 - Response JSON: success = true	- Status Code = 200 - Response JSON: success = true	✓	

12	Verify OTP Success: 1. Register ผู้ใช้ใหม่ 2. Login เพื่อรับ ACCESS_TOKEN 3. เรียก GET /api/otp/get-for-testing เพื่อดึง OTP 4. เรียก POST /api/otp/verify พร้อมส่ง otpCode 5. ตรวจสอบ Response	- Status Code = 200 - Response JSON มี key success	- Status Code = 200 - Response JSON มี key success	✓	
13	Delete Request Soft Delete: 1. Register ผู้ใช้ใหม่ 2. Login เพื่อรับ ACCESS_TOKEN 3. สร้าง Body: • deleteUserRequest = true • deleteVehicleRequest = true • deleteRouteRequest = true • deleteBookingRequest = true 4. เรียก POST /api/delete-request 5. ตรวจสอบ Response	- Status Code = 200 - Response JSON: success = true	- Status Code = 200 - Response JSON: success = true	✓	

CP353004/SC313004 Software Engineering

ผศ.ดร. ชิตสุธา สุ่มเล็ก ปีการศึกษา 2/2568

Test Scenario ID:	Unit016-1	**Project ID:**	PaiNamNae
Test Scenario Name:	Unit – Deletion service	**Tested by:**	ตะวัน อุตมาน
UAT Name:	Deletion Story	**Version:**	V1.0
Module:	deleteRequest.service.js	**Date of Test:**	03/03/2569
Pre-requisite:	-		
Description:			

No.	Test Case and Steps	Expected Result	Actual Result	Test Result (Pass/Fail)	Remark
14	Create Delete Request Success 1. Mock prisma.deleteRequest.create ให้ return { id: 1 } 2. เรียก createDeleteRequest(1, true, false, false, false) 3. ตรวจสอบผลลัพธ์	- มีการเรียก prisma.deleteRequest.create - Result = { id: 1 }	- มีการเรียก prisma.deleteRequest.create - Result = { id: 1 }	✓	
15	Get Delete Request Success: 1. Mock prisma.deleteRequest.findFirst ให้ return { id: 1 } 2. เรียก getDeleteRequestByUserId(1)	- Return { id: 1 }	- Return { id: 1 }	✓	
16	Get Delete Request Not Found: 1. Mock prisma.deleteRequest.findFirst ให้ return null 2. เรียก getDeleteRequestByUserId(1)	- Throw ApiError - Status = 404	- Throw ApiError - Status = 404	✓	
17	Active Route Exists: 1. Mock prisma.route.findFirst ให้ return { id: 99 } 2. เรียก checkCanDeleteAccount(1)	- result.canDelete = false - result.reason = "ACTIVE_ROUTE"	- result.canDelete = false - result.reason = "ACTIVE_ROUTE"	✓	

18	Active Booking Exists: 1. Mock route.findFirst = null 2. Mock booking.findFirst = { id: 10 } 3. เรียก checkCanDeleteAccount(1)	- result.canDelete = false - result.reason = "ACTIVE_BOOKING"	- result.canDelete = false - result.reason = "ACTIVE_BOOKING"	✓	
19	Can Delete Account: 1. Mock route.findFirst = null 2. Mock booking.findFirst = null 3. เรียก checkCanDeleteAccount(1)	- result.canDelete = true	- result.canDelete = true	✓	
20	Send Delete Request Success: 1. Mock route และ booking = null 2. Mock UserService.getUserById return mockUser 3. เรียก sendDeleteRequest({...}, 1) 4. ตรวจสอบการเรียก SoftDelete และ Email	- softDeleteUser(1) ถูกเรียก - sendEmail ถูกเรียก	- softDeleteUser(1) ถูกเรียก - sendEmail ถูกเรียก	✓	
21	Cannot Delete Account: 1. Mock route.findFirst return { id: 5 } 2. เรียก sendDeleteRequest({}, 1)	- Throw ApiError	- Throw ApiError	✓	

CP353004/SC313004 Software Engineering

ผศ.ดร. ชิตสุธา สุ่มเล็ก **ปีการศึกษา 2/2568**

Test Scenario ID:	Unit016-2	**Project ID:**	PaiNamNae
Test Scenario Name:	Unit – Email service	**Tested by:**	ตะวัน อุตมาน
UAT Name:	Deletion Story	**Version:**	V1.0
Module:	email.service.js	**Date of Test:**	03/03/2569
Pre-requisite:	- กำหนด process.env.EMAIL_USER - กำหนด process.env.EMAIL_PASS - Mock nodemailer		
Description:			

No.	Test Case and Steps	Expected Result	Actual Result	Test Result (Pass/Fail)	Remark
22	Send Email Success: 1. Mock nodemailer.__mockSendMail ให้ resolve(true) 2. สร้าง instance ของ EmailService 3. เรียก sendEmail() พร้อม parameter: • to • subject • text • attachments = [] 4. ตรวจสอบการเรียก mock function	- __mockSendMail ถูกเรียก 1 ครั้ง - ไม่มี error thrown	- __mockSendMail ถูกเรียก 1 ครั้ง - ไม่มี error thrown	✓	
23	Send Email Fail: 1. Mock nodemailer.__mockSendMail ให้ reject(Error: "SMTP error") 2. สร้าง instance EmailService 3. เรียก sendEmail()	- Throw ApiError - Status Code = 500	- Throw ApiError - Status Code = 500	✓	

Test Scenario ID:	Unit016-3	**Project ID:**	PaiNamNae
Test Scenario Name:	Unit – OTP service	**Tested by:**	ตะวัน อุตมาน
UAT Name:	Deletion Story	**Version:**	V1.0
Module:	otp.service.js	**Date of Test:**	03/03/2569
Pre-requisite:	- Mock prisma.otp (findFirst, deleteMany, create) - Mock crypto.randomInt - Mock emailService.sendEmail		
Description:			

No.	Test Case and Steps	Expected Result	Actual Result	Test Result (Pass/Fail)	Remark
24	Get OTP Found: 1. Mock prisma.otp.findFirst return { id:1, email } 2. เรียก getOtpNotExpireByEmail(email)	- Return OTP object - ไม่เกิด error	- Return OTP object - ไม่เกิด error	✓	
25	Get OTP Not Found: 1. Mock findFirst return null 2. เรียก getOtpNotExpireByEmail(email)	- Return null	- Return null	✓	
26	Delete OTP By ID: 1. เรียก deleteOtpById(1)	- เรียก prisma.otp.deleteMany - where: { id: 1 }	- เรียก prisma.otp.deleteMany - where: { id: 1 }	✓	
27	Delete Expired OTPs: 1. เรียก deleteExpiredOtps()	- เรียก prisma.otp.deleteMany	- เรียก prisma.otp.deleteMany	✓	
28	Create OTP Success: 1. Mock findFirst = null 2. Mock randomInt return 123456 3. เรียก createOtp(email)	- เรียก prisma.otp.create - เรียก sendEmail 1 ครั้ง	- เรียก prisma.otp.create - เรียก sendEmail 1 ครั้ง	✓	

30	Replace Old OTP: 1. Mock findFirst return { id: 99 } 2. เรียก createOtp(email)	- เรียก deleteMany({ where: { id: 99 } }) - เรียก create	- เรียก deleteMany({ where: { id: 99 } }) - เรียก create	✓	
31	Verify OTP Success: 1. Mock findFirst return { id:1, otpCode:123456 } 2. เรียก verifyOtp(email, 123456)	- Return true - เรียก deleteMany({ where: { id:1 } })	- Return true - เรียก deleteMany({ where: { id:1 } })	✓	
32	Expired or Not Found: 1. Mock findFirst return null 2. เรียก verifyOtp(email, otp)	- Throw Error: "OTP ไม่ถูกต้องหรือหมดอายุแล้ว"	- Throw Error: "OTP ไม่ถูกต้องหรือหมดอายุแล้ว"	✓	
33	OTP Not Match: 1. Mock findFirst return otpCode ≠ input 2. เรียก verifyOtp(email, otp)	- Throw Error: "OTP ไม่ถูกต้อง"	- Throw Error: "OTP ไม่ถูกต้อง"	✓	

5. รายงานสรุปผลการทดสอบ (Test Summary Report)

Scenario ID	Scenario Name	Test Case#	Pass	Fail	No run	Block	Remark	Defect ID
UAT016-1	UAT - ลบ Account สำเร็จ	2	2	0	0	0		
UAT016-2	UAT - ลบ Account ไม่สำเร็จ	4	2	0	0	0		
UAT016-3	UAT - ทดสอบ checkboxes	2	2	0	0	0		
API016	API - Account Deletion	5	5	0	0	0		
Unit016-1	Unit – Deletion service	8	8	0	0	0		
Unit016-2	Unit – Email service	2	2	0	0	0		
Unit016-3	Unit – OTP service	9	9	0	0	0		
รวม		32	32	0	0	0		